

GHANA INSTITUTE OF MANAGEMENT AND PUBLIC
ADMINISTRATION

School of Technology | Department of Computer Science

CS 305: Theory of Computation

Level 300 Undergraduate | Syllabus, 15-Week Schedule & Applied Modern Case
Studies

Course Specifications			
Course Title:	Theory of Computation	Academic Level:	Level 300 (BSc CS / IT)
Course Code:	CS 305 / SOT 305	Primary Text:	Michael Sipser (3rd Ed.)
Course Credit:	3 Credit Hours	Prerequisites:	Discrete Mathematics
Office Hours:	By Appointment	Target Term:	Semester / Trimester Delivery

1. Course Overview & Modern Relevance

What can a computer solve, and what are the fundamental, unbreakable mathematical limits of computation? **CS 305: Theory of Computation** is the foundational bedrock of computer science. It establishes formal mathematical abstractions of computing machines to answer two grand questions: *What is computable?* (*Computability Theory*) and *What is computable efficiently in polynomial time?* (*Complexity Theory*).

Rather than treating theory as isolated historical mathematics, this course deliberately bridges classical foundational theorems (by Turing, Chomsky, and Cook) to **modern 2026 computer science systems**:

- (a) Automata & Formal Languages:** Finite state machines, regular expressions, context-free grammars, and pushdown automata — with direct applications to **compiler lexical scanning**, **network intrusion detection (Snort/Zeek)**, and **LLM structured JSON generation (grammar-guided decoding)**.
- (b) Computability Theory:** Turing machines, the Church–Turing thesis, decidability, the Halting Problem, and Rice’s Theorem — with direct applications to **Ethereum smart contract gas mechanics**, **impossibility of infallible anti-virus engines**, and **Chain-of-Thought in Large Language Models**.
- (c) Complexity Theory:** Time complexity, classes **P** vs. **NP**, and polynomial-time reductions — with direct applications to **modern cryptography (RSA, Elliptic Curves, Post-Quantum Lattices)** and **industrial SAT/SMT solvers (Z3)**.

2. Prerequisites

Required Background Knowledge
Discrete Mathematics (CS 201): A solid foundation in set theory, binary relations, functions, mathematical induction, and basic proof techniques (direct proof, proof by contradiction, contraposition).

3. Prescribed Textbooks & Software Tools

- Primary Textbook:** Michael Sipser, *Introduction to the Theory of Computation*, 3rd International Edition, Cengage Learning. (The global gold standard used at Stanford, MIT, and UC Berkeley).
- Interactive Simulators:** JFLAP (www.jflap.org) and Automaton Simulator (automatonsimulator.com) for visual DFA/NFA/PDA and Turing Machine execution.
- Programming Languages for Labs:** Python 3.10+ (for automata simulation and SAT reductions).

4. Course Learning Outcomes (CLOs)

By the end of this course, Level 300 students will be able to:

- ✓ **CLO 1:** Formally construct, trace, and minimize Deterministic and Nondeterministic Finite Automata (DFAs/NFAs), Regular Expressions, and Context-Free Grammars (CFGs).
- ✓ **CLO 2:** Apply the Pumping Lemma to rigorously prove non-regularity and non-context-freeness of formal languages.
- ✓ **CLO 3:** Design Turing Machines, trace configuration histories, and explain the Church–Turing Thesis.
- ✓ **CLO 4:** Formulate undecidability proofs using Cantor’s diagonalization argument and formal mapping reductions ($\leq_m$).
- ✓ **CLO 5:** Classify computational problems into time complexity classes (**P** and **NP**) and prove **NP**-completeness via polynomial-time reductions ($\leq_P$).
- ✓ **CLO 6:** Connect theoretical computability and complexity limits to modern computer systems, including LLM grammar-constrained decoding, smart contract execution models, cybersecurity verification limits, and public-key cryptography.

5. Assessment Breakdown

Evaluation combines continuous biweekly problem sets, short quizzes, mid-semester assessment, an independent applied term project, and a comprehensive final exam:

Assessment Component	Format & Frequency	Weight (%)
Problem Sets	4 biweekly homework sets (7.5% each)	30%
Quizzes	2 short in-class diagnostic quizzes (5% each)	10%
Midterm Examination	1 written mid-trimester exam (Week 8)	20%
Term Project / Demo	Independent reduction paper or visual TM simulator	10%
Final Examination	Comprehensive end-of-trimester exam	30%
Total		100%

6. 15-Week Detailed Schedule & Modern Technology Spotlights

Wk	Topic & Content	Sipser	Modern Spotlight & Applied Case Study	Milestones
1	Preliminaries & Computation Models Alphabets, strings, formal languages, proof techniques	Ch. 0	Chomsky Hierarchy in Modern AI: Why finite models cannot parse recursive language; LLM tokenization alphabets	Diagnostic Math Review
2	Deterministic Finite Automata DFA 5-tuple, state diagrams, formal language acceptance	1.1	Network Packet Filtering & Compilers: How Snort/Zeek firewalls and compiler lexers scan millions of tokens in $\mathcal{O}(n)$ time	Problem Set 1 Assigned
3	Nondeterministic Finite Automata NFA definition, non-determinism, DFA/NFA equivalence	1.2	NFA State Explosion in Search Engines: How grep and search indexing simulate multiple hypotheses simultaneously	Problem Set 1 Due
4	Regular Expressions & GNFA Regular operations, equivalence with finite automata	1.3	Catastrophic ReDoS Cloud Outages: How exponential backtracking in non-DFA regex engines crashed Cloudflare in 2019	Quiz 1
5	Nonregular Languages Limits of finite automata; Pumping Lemma for regular languages	1.4	Memory Bounds of Finite State Systems: Why a DFA can count to 1 billion but cannot verify arbitrary matched brackets	Problem Set 2 Assigned
6	Context-Free Grammars CFG definitions, derivations, parse trees, ambiguity	2.1	Grammar-Guided LLM JSON Generation: How APIs (Outlines/Guidance) force LLMs to output valid JSON/SQL via CFG masks	Problem Set 2 Due

Wk	Topic & Content	Sipser	Modern Spotlight & Applied Case Study	Milestones
7	Pushdown Automata PDA definition, stack memory, equivalence of PDAs and CFGs	2.2	Compiler ASTs & Runtime Call Stacks: Why stack memory is strictly required for nested expressions and syntax trees	Midterm Review
8	Midterm Exam & Non-CFLs Pumping Lemma for Context-Free Languages	2.3	Transformer Limits on Dyck Languages: Why standard attention mechanisms struggle with deep recursive parentheses	Midterm Examination
9	Turing Machines Formal 7-tuple, tape head, configurations, TM variants	3.1–3.2	Chain-of-Thought LLMs as Turing Machines: Why an autoregressive LLM with an external scratchpad tape becomes Turing-complete	Problem Set 3 Assigned (Project Announced)
10	Church–Turing Thesis Equivalence of TM variants; formal algorithm definition	3.3	Physical Limits of Computation: Quantum computers, biological computing, and why no physical machine exceeds the TM	Problem Set 3 Due
11	Decidability Decidable languages, decision procedures for regular/CFL problems	4.1	Automated Software Verification: How static analysis tools prove memory safety and deadlock freedom in critical software	Quiz 2
12	Undecidability & Halting Problem A_{TM} , Cantor’s diagonalization, mapping reductions, Rice’s theorem	4.2, 5.1	Smart Contract Gas & Anti-Virus Limits: (1) Why Ethereum charges Gas (Halting Problem); (2) Why an infallible anti-virus is impossible (Rice)	Problem Set 4 Assigned
13	Time Complexity & Class P Measuring time complexity, asymptotic bounds, class P	7.1–7.2	Polynomial vs Exponential in Production: Why $\mathcal{O}(n^k)$ algorithms scale to global web scale while $\mathcal{O}(2^n)$ algorithms stall	Problem Set 4 Due
14	Class NP & NP-Completeness Class NP, Cook–Levin theorem, 3SAT, CLIQUE reductions	7.3–7.4	Post-Quantum Cryptography & SAT Solvers: How industrial solvers (Z3) solve 1M-variable NP-hard problems; RSA & Lattice crypto	Term Project Demos
15	Course Synthesis & Quantum Teaser P vs NP Millennium Prize; intro to quantum class BQP	Review	The Future of Computation: Shor’s algorithm, quantum supremacy, and open frontiers in theoretical computer science	Final Examination

7. Pedagogical & Proof-Writing Guidance for Students

Mastering Theoretical Proofs

- **The Pumping Lemma Strategy (Weeks 5 & 8):** Frame the Pumping Lemma as an adversarial game between you and an opponent:

Opponent gives $p \rightarrow$ **You** choose $s \in L$ ($|s| \geq p$) $\rightarrow$ **Opponent** splits $s = xyz \rightarrow$ **You** pick i such that $xy^iz \notin L$
- **Mapping Reductions ($A \leq_m B$):** To prove language A is undecidable using known undecidable language B , remember the direction of reduction: **Reduce the known hard problem to the unknown problem** ($A_{TM} \leq_m L$).
- **Term Project Options (Due Week 14):**
 - (a) *Implementation Track:* Build an interactive visual simulator for DFAs/NFAs or a Multi-Tape Turing Machine in Python/JavaScript.
 - (b) *Applied Case Study Track:* Write a 5-page research paper analyzing a modern system through theoretical lenses (e.g., ReDoS analysis of popular npm libraries, or modeling smart contract execution limits).

8. Prescribed Contemporary Research Papers by Lecture Module

To bridge theoretical rigor with modern breakthroughs, students will read selected seminal and contemporary research papers from top computer science venues (**ICLR, NeurIPS, USENIX Security, ACM CCS, POPL, EMNLP**):

- [1] **Week 1 (Chomsky Hierarchy & LLMs):** G. Delétang, et al., “*Neural Networks and the Chomsky Hierarchy*,” **International Conference on Learning Representations (ICLR 2023)**, Spotlight Paper.
- [2] **Weeks 2–3 (DFAs in High-Speed Networks):** S. Kumar, et al., “*Algorithms to Accelerate Multiple Regular Expression Matching for Deep Packet Inspection*,” **ACM SIGCOMM Computer Communication Review**, 2006.
- [3] **Week 4 (Catastrophic ReDoS Cloud Attacks):** C.-A. Staicu and M. Pradel, “*Freezing the Web: A Study of ReDoS Vulnerabilities in JavaScript-based Web Servers*,” **USENIX Security Symposium**, 2018.
- [4] **Week 6 (CFGs & Structured LLM Generation):** B. T. Willard and R. Louf, “*Efficient Guided Generation for Large Language Models*,” **NeurIPS Foundation Models Workshop**, 2023.
- [5] **Week 8 (Dyck Languages & Attention Limits):** M. Hahn, “*Theoretical Limitations of Self-Attention in Neural Sequence Models*,” **Transactions of the Association for Computational Linguistics (TACL)**, 2020.
- [6] **Week 9 (Turing Completeness of Chain-of-Thought):** W. Merrill and A. Sabharwal, “*The Expressive Power of Transformers with Chain of Thought*,” **International Conference on Learning Representations (ICLR 2024)**.
- [7] **Week 12 (Undecidability & Smart Contract Security):** L. Luu, et al., “*Making Smart Contracts Smarter*,” **ACM Conference on Computer and Communications Security (CCS)**, 2016.
- [8] **Week 12 (Rice’s Theorem & Malware Limits):** F. Cohen, “*Computer Viruses: Theory and Experiments*,” **Computers & Security**, 1987.
- [9] **Week 14 (Automated SAT Solvers in Industry):** L. de Moura and N. Bjørner, “*Z3: An Efficient SMT Solver*,” **International Conference on Tools and Algorithms for the Construction and**

Analysis of Systems (TACAS / Springer LNCS), 2008.

- [10] **Week 15 (Post-Quantum Cryptography):** E. Alkim, et al., “*Post-Quantum Key Exchange: A New Hope*,” **USENIX Security Symposium**, 2016.